Lab#07
Implementation of Stack as an Array & Linkedlist

A **Stack** is a linear data structure that follows the Last-In-First-Out (LIFO) principle. It can be implemented using an array by treating the end of the array as the top of the stack.

Declaration of Stack using Array
A stack can be implemented using an array where we maintain:

- An integer array to store elements.
- A variable capacity to represent the maximum size of the stack.
- A variable top to track the index of the top element. Initially, top = -1 to indicate an empty stack.

In computer science, a stack is an abstract data type that serves as a collection of elements, with two main operations:

Push, which adds an element to the collection, and
Pop, which removes the most recently added element that was not yet removed.

Stacks entered the computer science literature in 1946, when Alan M. Turing used the terms "bury" and "unbury" as a means of calling and returning from subroutines. Stack follows LIFO data structure type.

Operations On Stack

Push Operation:
Adds an item to the stack. If the stack is full, then it is said to be an Overflow condition.

- Before pushing the element to the stack, we check if the stack is full.
- If the stack is full (top == capacity-1) , then Stack Overflows and we cannot insert the element to the stack.
- Otherwise, we increment the value of top by 1 (top = top + 1) and the new value is inserted at top position .
- The elements can be pushed into the stack till we reach the capacity of the stack.

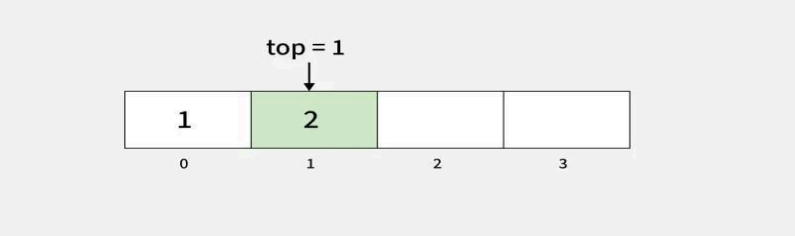
Pop Operation:
Removes an item from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an Underflow condition.

Before popping the element from the stack, we check if the stack is empty .

If the stack is empty (top == -1), then Stack Underflows and we cannot remove any element from the stack.
Otherwise, we store the value at top, decrement the value of top by 1 (top = top – 1) and return the stored top value.

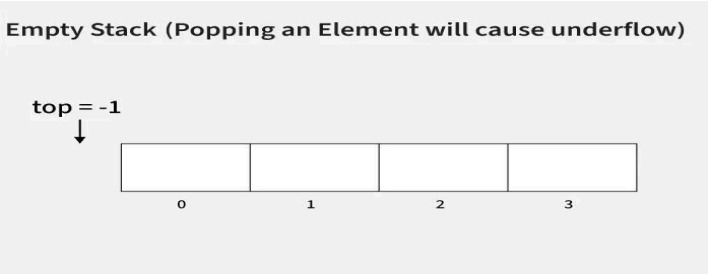
Top or Peek Operation in Stack:
Returns the top element of the stack.

- Before returning the top element from the stack, we check if the stack is empty.
- If the stack is empty (top == -1), we simply print “Stack is empty”.
- Otherwise, we return the element stored at index = top.



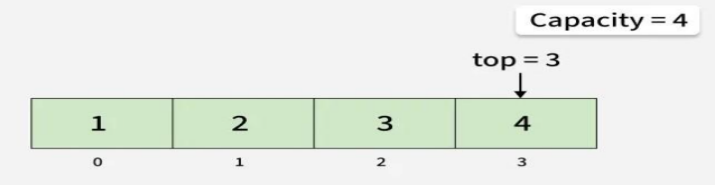
isEmpty Operation in Stack:
Returns true if the stack is empty, else false.

- Check for the value of top in stack.
- If (top == -1) , then the stack is empty so return true.
- Otherwise, the stack is not empty so return false.



isFull Operation in Stack :
Returns true if the stack is full, else false.

- Check for the value of top in stack.
- If (top == capacity-1), then the stack is full so return true.
- Otherwise, the stack is not full so return false.



Stack Implementation using Dynamic Array
When using a fixed-size array, the stack has a maximum capacity that cannot grow beyond its initial size. To overcome this limitation, we can use dynamic arrays. Dynamic arrays automatically resize themselves as elements are added or removed, which makes the stack more flexible.

In C++, we can use vector.
In Java, we can use ArrayList.
In Python, we can use list.
In C#, we can use List.

Implementation of Stack in LinkedList

A stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle. It can be implemented using a linked list, where each element of the stack is represented as a node. The head of the linked list acts as the top of the stack.

Declaration of Stack using Linked List
A stack can be implemented using a linked list where we maintain:

- A Node structure/class that contains:
  data → to store the element.
  next → pointer/reference to the next node in the stack.
- A pointer/reference top that always points to the current top node of the stack.
  Initially, top = null to represent an empty stack.

In computer science, a stack is an abstract data type that serves as a collection of elements, with two main operations:

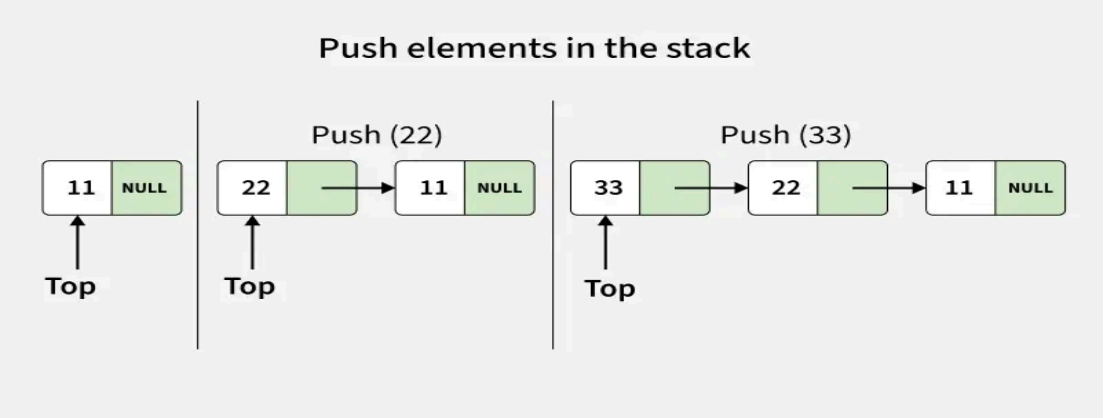
Push, which adds an element to the collection, and
Pop, which removes the most recently added element that was not yet removed.

Stacks entered the computer science literature in 1946, when Alan M. Turing used the terms "bury" and "unbury" as a means of calling and returning from subroutines. Stack follows LIFO data structure type.

Operations on Stack using Linked List

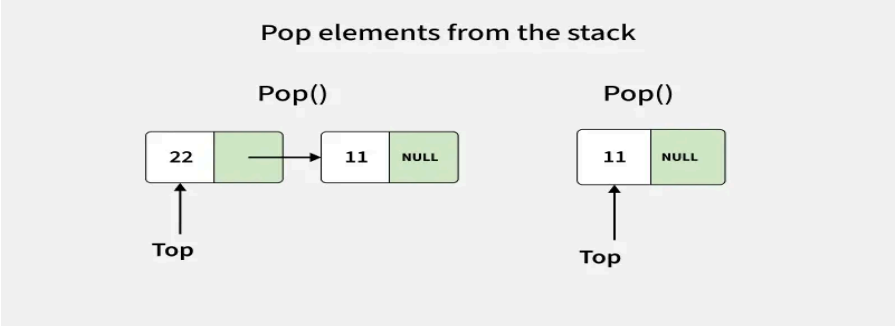
Push Operation
Adds an item to the stack. Unlike array implementation, there is no fixed capacity in linked list. Overflow occurs only when memory is exhausted.

- A new node is created with the given value.
- The new node’s next pointer is set to the current top.
- The top pointer is updated to point to this new node.



Pop Operation
Removes the top item from the stack. If the stack is empty, it is said to be an Underflow condition.

- Before deleting, we check if the stack is empty (top == NULL).
- If the stack is empty, underflow occurs and deletion is not possible.
- Otherwise, we store the current top node in a temporary pointer.
- Move the top pointer to the next node.
- Delete the temporary node to free memory.



Peek (or Top) Operation
Returns the value of the top item without removing it from the stack.

- If the stack is empty (top == NULL), then no element exists.
- Otherwise, simply return the data of the node pointed by top.

isEmpty Operation
Checks whether the stack has no elements.

- If the top pointer is NULL, it means the stack is empty and the function returns true.
- Otherwise, it returns false.

Lab Tasks

1. Write a C++ program using (a**) arrays** (b)**linked list** to implement a stack using an array with push and pop operations. Find the top element of the stack and check if the stack is empty or not.

2. Write a C++ program to implement a stack using (a) **arrays**(b)**linked list** with push and pop operations. Check if the stack is full.

3. Write a C++ program to sort a given stack (using an array) using another stack.

Input some elements onto the stack:
Stack elements: 0 1 5 2 4 7
Sort the elements in the stack:
Display the sorted elements of the stack:
Stack elements: 0 1 2 4 5 7
Remove two elements:
Stack elements: 2 4 5 7

Input two more elements
Stack elements: 10 -1 2 4 5 7
Sort the elements in the stack:
Display the sorted elements of the stack:
Stack elements: -1 2 4 5 7 10

4. Write a C++ program using (a) **arrays** (b)**linked list** that reverses the stack (using an array) elements.

5. Write a C++ program using (a) **arrays** (b)**linked list** to find and remove the lowest element in a stack.